

ChatGPT - Mid

Question 1

a) How do you implement many to many cardinality?
Explain with a suitable example.

b) What is a Tablespace? Mention one scenario where it is essential to bind tablespace to a specific table.

c) "It is possible to convert each PL/SQL procedure into an equivalent function." Do you agree with the statement? Justify your position using suitable arguments.

Answer the questions in details and clearly

Certainly! Here are detailed, clear answers for each part of the question:

1a) How do you implement many-to-many cardinality? Explain with a suitable example.

Explanation

A **many-to-many relationship** between two entities means that one row in the first table can relate to many rows in the second table, and one row in the second table can relate to many rows in the first table.

Relational databases do **not** support many-to-many relationships directly.

Instead, we **implement** it using an **associative (junction) table** (also called a bridge table or linking table).

How to implement:

1. **Create the two main tables** (let's say, Students and Courses).

2. **Create a third table** (e.g., Enrollments) that holds the primary keys of both main tables as foreign keys.

Example:

Suppose you have:

- **Students:** A student can enroll in many courses.
- **Courses:** Each course can have many students.

Tables:

sql

```
CREATE TABLE Students (  
  student_id INT PRIMARY KEY,  
  name VARCHAR(100)  
);
```

```
CREATE TABLE Courses (  
  course_id INT PRIMARY KEY,  
  course_name VARCHAR(100)  
);
```

-- This is the junction/association table

```
CREATE TABLE Enrollments (  
  student_id INT,  
  course_id INT,  
  enrollment_date DATE,  
  PRIMARY KEY (student_id, course_id),  
  FOREIGN KEY (student_id) REFERENCES Students(student_id),  
  FOREIGN KEY (course_id) REFERENCES Courses(course_id)  
);
```

How it works:

- If student 1 enrolls in course A and B, and student 2 enrolls in course A:
 - Enrollments table will have:
 - (1, A)
 - (1, B)
 - (2, A)

Summary Table:

student_id	course_id	enrollment_date
1	A	2023-07-01
1	B	2023-07-02
2	A	2023-07-01

Key Point:

The **Enrollments** table breaks down the many-to-many relationship into two one-to-many relationships (Student to Enrollment, Course to Enrollment).

1b) What is a Tablespace? Mention one scenario where it is essential to bind tablespace to a specific table.

Explanation

A **tablespace** is a logical storage unit in a database that groups related logical structures together, such as tables and indexes.

- It is used mainly in Oracle and similar databases.

Think of tablespaces as containers for database objects.

- Physically, a tablespace consists of one or more data files on disk.
- Logically, it is used to organize and manage data storage in the database.

Scenario where binding a table to a specific tablespace is essential:

Scenario:

Suppose you are managing a very large database. Some tables grow very fast (e.g., transaction logs, audit trails), while others remain relatively small.

Reasons to bind a table to a specific tablespace:

- **Performance:** Place high-usage tables on fast disks by assigning them to a dedicated tablespace.
- **Maintenance:** Backup, restore, or manage only that tablespace (without affecting others).

- **Separation:** Store historical or archive data in a separate tablespace for better organization.
- **Security:** Sensitive tables can be stored in encrypted tablespaces.

Example:

```
sql

CREATE TABLE audit_log (
  id NUMBER PRIMARY KEY,
  action VARCHAR2(100),
  action_time DATE
) TABLESPACE audit_ts;
```

Here, the **audit_log** table is stored in the **audit_ts** tablespace, which may be on a separate, high-capacity, secure disk.

1c) "It is possible to convert each PL/SQL procedure into an equivalent function." Do you agree with the statement? Justify your position using suitable arguments.

Explanation

PL/SQL Procedures and Functions are both named PL/SQL blocks, but there is a fundamental difference:

- **Procedure:** Performs an action. May or may not return a value. Called as a standalone PL/SQL statement.
- **Function:** Must return a value using the RETURN keyword. Can be used in SQL expressions (if they meet certain criteria).

Argument:

I do NOT fully agree with the statement that every procedure can be converted into an equivalent function.

Reasons:

1. Purpose & Design:

- Functions are designed to compute and return a value.
- Procedures are designed to perform an operation (may involve DML, complex logic, etc.), and returning a value is optional.

2. Usage in SQL:

- Only functions (with certain restrictions) can be called directly inside SQL statements.
- Procedures cannot be used in SQL queries.

3. Multiple Outputs:

- Procedures can return multiple values using OUT parameters.
- Functions return only **one** value (although you can return a record or collection, it's still a single object).

4. Restrictions:

- Functions called in SQL should not modify database state (e.g., cannot perform INSERT/UPDATE/DELETE). Procedures do not have this restriction.

5. Side Effects:

- Procedures are used when side effects (such as modifying tables) are required.
- Functions are supposed to be free of side effects when used in SQL.

Example:

Procedure with multiple OUT parameters:

```
sql

CREATE PROCEDURE get_employee_details (
  emp_id IN NUMBER,
  emp_name OUT VARCHAR2,
  emp_salary OUT NUMBER
)
AS
BEGIN
  SELECT name, salary INTO emp_name, emp_salary FROM employees WHERE id = emp_id;
END;
```

Cannot be simply converted to a function, because a function can only return one value.

Conclusion:

- **Not every procedure can be meaningfully or safely converted into a function.**
- **It depends on the logic and intended use.**
- Functions and procedures serve different purposes in PL/SQL, and their usage depends on the requirement.

Summary Table

Point	Procedures	Functions
Purpose	Perform action(s)	Compute and return a value
Return Value	Optional (via OUT/INOUT)	Mandatory (via RETURN)
Use in SQL	No	Yes (if certain restrictions are met)
Side Effects	Allowed	Not allowed in SQL queries
Multiple Output	Yes (via OUT parameters)	No (only one return value)

In summary:

- **a)** Many-to-many relationships are implemented via a junction table.
- **b)** Tablespaces logically group storage; binding tables to specific tablespaces is crucial for performance, maintenance, and organization.
- **c)** Not every PL/SQL procedure can be converted into an equivalent function because they have different design purposes and usage constraints.

Question 2

Scenario:

Students of IUT are admitted in different programs of each department. Each student must have ID, Name, Date of Birth, CGPA, and his/her department/program information.

Pocket Allowance Selection Policy:

Students are given a monthly Pocket Allowance (PA) of 50 US\$ which is reflected through a separate transaction containing the student's ID, amount, and date-time. The office of the registrar maintains disciplinary record for each student as well. If he/she has a CGPA ≤ 2.25 and has 3 or more disciplinary events recorded in the last 6 months, then the student is disqualified from getting PA. The university also provides internet service of 100 hours per month for each student. Beyond this limit, charge will be applied as follows:

Day Scheme: The charge from 6:00 up to 18:00 is 0.10 US\$ per minute.

Night Scheme: The charge from 18:01 up to 5:59 is 0.25 US\$ per minute.

Starting time of a specific internet connection will be the deciding factor for its specific scheme. In both cases, 1 minute pulse is considered (i.e., 1 minute 12 seconds will be considered as 2 minutes).

Instead of paying separately, the bills (if any) are adjusted with their PA (i.e., deducted).

Tasks:

a) Create the required table(s) maintaining standard database design principles.

b) Write a PL/SQL function as follows:

Input: ID, Current Date-Time

Output: Yes/No

Algorithm: It determines if the student is allowed to receive PA as per the policy

c) Write a PL/SQL function as follows:

Input: ID, Current Date-Time

Output: Amount of total bill

Algorithm: It calculates total bill for given student for the given month as per the policy

Solve it in details with clear explanation of everything.
explain everything to me and the steps to solve it

Absolutely! Let's break down each part step by step, with clear explanations and reasoning for the database design, logic, and PL/SQL functions.

a) Database Table Design

Let's identify the required **entities** and their attributes, then design normalized tables.

Entities Identified from Scenario

1. Student

- ID, Name, Date of Birth, CGPA, Department, Program

2. Department (with multiple programs per department)

- Dept_ID, Dept_Name

3. Program

- Program_ID, Program_Name, Dept_ID

4. Pocket Allowance Transaction

- Transaction_ID, Student_ID, Amount, DateTime

5. Disciplinary Record

- Record_ID, Student_ID, Event_Date, Description

6. Internet Usage

- Usage_ID, Student_ID, Start_DateTime, Duration_Minutes

Table Design (with sample attributes and constraints)

1. Department Table

```
sql
```



```
CREATE TABLE Department (  
  Dept_ID NUMBER PRIMARY KEY,  
  Dept_Name VARCHAR2(100) NOT NULL  
);
```

2. Program Table

sql

```
CREATE TABLE Program (  
  Program_ID NUMBER PRIMARY KEY,  
  Program_Name VARCHAR2(100) NOT NULL,  
  Dept_ID NUMBER NOT NULL,  
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)  
);
```

3. Student Table

sql

```
CREATE TABLE Student (  
  Student_ID NUMBER PRIMARY KEY,  
  Name VARCHAR2(100) NOT NULL,  
  DOB DATE NOT NULL,  
  CGPA NUMBER(3,2) NOT NULL,  
  Dept_ID NUMBER NOT NULL,  
  Program_ID NUMBER NOT NULL,  
  FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID),  
  FOREIGN KEY (Program_ID) REFERENCES Program(Program_ID)  
);
```

4. Pocket Allowance Transaction Table

sql

```
CREATE TABLE PA_Transaction (  
  Transaction_ID NUMBER PRIMARY KEY,  
  Student_ID NUMBER NOT NULL,  
  Amount NUMBER(6,2) NOT NULL,  
  Transaction_Date DATE NOT NULL,  
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)  
);
```

5. Disciplinary Record Table

sql

```
CREATE TABLE Disciplinary_Record (  
  Record_ID NUMBER PRIMARY KEY,  
  Student_ID NUMBER NOT NULL,  
  Event_Date DATE NOT NULL,  
  Description VARCHAR2(200),  
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)  
);
```

6. Internet Usage Table

sql

```
CREATE TABLE Internet_Usage (  
  Usage_ID NUMBER PRIMARY KEY,  
  Student_ID NUMBER NOT NULL,  
  Start_DateTime DATE NOT NULL,  
  Duration_Minutes NUMBER NOT NULL, -- Always rounded up  
  FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)  
);
```

Notes:

- All foreign keys enforce referential integrity.
- No repeating groups, and tables are in at least 3NF.
- No redundancy: program and department separated, events and transactions separated.

b) PL/SQL Function: Check Pocket Allowance Eligibility

Goal:

Given a Student_ID and current date, return 'Yes' if student is eligible for PA, else 'No' as per the rules:

- CGPA > 2.25 **OR**
- Disciplinary events in last 6 months < 3

Algorithm Steps

1. Get student's CGPA.
2. Count disciplinary events for this student in the last 6 months from current date.
3. If $CGPA > 2.25$ OR $events < 3 \rightarrow$ eligible ('Yes'), else ineligible ('No').

PL/SQL Function

sql

```
CREATE OR REPLACE FUNCTION Is_Eligible_For_PA(p_Student_ID IN NUMBER, p_Current_Date IN
DATE)
RETURN VARCHAR2
IS
    v_CGPA Student.CGPA%TYPE;
    v_Event_Count NUMBER;
BEGIN
    -- 1. Get the student's CGPA
    SELECT CGPA INTO v_CGPA
    FROM Student
    WHERE Student_ID = p_Student_ID;

    -- 2. Count disciplinary events in the last 6 months
    SELECT COUNT(*) INTO v_Event_Count
    FROM Disciplinary_Record
    WHERE Student_ID = p_Student_ID
    AND Event_Date >= ADD_MONTHS(p_Current_Date, -6);

    -- 3. Apply the policy
    IF v_CGPA > 2.25 OR v_Event_Count < 3 THEN
        RETURN 'Yes';
    ELSE
        RETURN 'No';
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN
        RETURN 'No'; -- Student does not exist or no records found
END;
/
```

Explanation

- `ADD_MONTHS(p_Current_Date, -6)` gives the date six months ago.
 - Returns 'Yes' if the student qualifies, otherwise 'No'.
 - Exception handling: returns 'No' if student not found.
-

c) PL/SQL Function: Calculate Internet Bill for a Student (Per Month)

Goal:

Given `Student_ID` and current date-time, calculate bill for current month if usage exceeds 100 hours.

Algorithm Steps

1. Find all internet usage records for this student in this month.
2. Sum up the total minutes.
3. If total minutes ≤ 6000 (100 hours $\times 60$), bill is \$0.
4. If more, bill for extra minutes using the pulse rule and correct day/night scheme.
 - **Day Scheme (6:00–18:00):** \$0.10/min
 - **Night Scheme (18:01–5:59):** \$0.25/min
 - Scheme decided by the start time of each session.
5. Sum the bill for extra minutes **only** (minutes beyond 6000).

PL/SQL Function

sql

```
CREATE OR REPLACE FUNCTION Calc_Internet_Bill(p_Student_ID IN NUMBER, p_Current_Date IN
DATE)
RETURN NUMBER
IS
    CURSOR usage_cur IS
        SELECT Start_DateTime, Duration_Minutes
        FROM Internet_Usage
        WHERE Student_ID = p_Student_ID
        AND EXTRACT(YEAR FROM Start_DateTime) = EXTRACT(YEAR FROM p_Current_Date)
```

```
AND EXTRACT(MONTH FROM Start_DateTime) = EXTRACT(MONTH FROM p_Current_Date);
```

```
v_Total_Minutes NUMBER := 0;
```

```
v_Bill NUMBER := 0;
```

```
v_Extra_Minutes NUMBER := 0;
```

```
v_Usage_Start_Hour NUMBER;
```

```
v_Minutes_To_Bill NUMBER;
```

```
v_Remaining_Free NUMBER := 6000; -- 100 hours = 6000 min
```

```
BEGIN
```

```
-- 1. Loop through all usage records this month
```

```
FOR usage_rec IN usage_cur LOOP
```

```
-- 1 minute pulse (round up to next minute if not exact)
```

```
v_Minutes_To_Bill := CEIL(usage_rec.Duration_Minutes);
```

```
v_Total_Minutes := v_Total_Minutes + v_Minutes_To_Bill;
```

```
END LOOP;
```

```
-- 2. Calculate extra minutes
```

```
IF v_Total_Minutes > v_Remaining_Free THEN
```

```
v_Extra_Minutes := v_Total_Minutes - v_Remaining_Free;
```

```
ELSE
```

```
v_Extra_Minutes := 0;
```

```
END IF;
```

```
-- 3. Now loop again to bill only extra minutes in order
```

```
v_Total_Minutes := 0;
```

```
FOR usage_rec IN usage_cur LOOP
```

```
v_Minutes_To_Bill := CEIL(usage_rec.Duration_Minutes);
```

```
v_Usage_Start_Hour := TO_NUMBER(TO_CHAR(usage_rec.Start_DateTime, 'HH24'));
```

```
-- Only charge if still extra minutes left
```

```
IF v_Extra_Minutes > 0 THEN
```

```
-- Find billable minutes for this session (might be partially billable)
```

```
IF v_Total_Minutes + v_Minutes_To_Bill <= v_Remaining_Free THEN
```

```
-- All minutes of this session are free
```

```
NULL;
```

```
ELSIF v_Total_Minutes >= v_Remaining_Free THEN
```

```
-- All minutes billable
```

```
IF v_Usage_Start_Hour >= 6 AND v_Usage_Start_Hour < 18 THEN
```

```
v_Bill := v_Bill + (v_Minutes_To_Bill * 0.10);
```

```
ELSE
```

```
v_Bill := v_Bill + (v_Minutes_To_Bill * 0.25);
```

```

END IF;
v_Extra_Minutes := v_Extra_Minutes - v_Minutes_To_Bill;
ELSE
  -- Partial billable (session crosses free limit)
  v_Session_Billable := v_Total_Minutes + v_Minutes_To_Bill - v_Remaining_Free;
  IF v_Usage_Start_Hour >= 6 AND v_Usage_Start_Hour < 18 THEN
    v_Bill := v_Bill + (v_Session_Billable * 0.10);
  ELSE
    v_Bill := v_Bill + (v_Session_Billable * 0.25);
  END IF;
  v_Extra_Minutes := v_Extra_Minutes - v_Session_Billable;
END IF;
END IF;
v_Total_Minutes := v_Total_Minutes + v_Minutes_To_Bill;
END LOOP;

RETURN v_Bill;
END;
/

```

Explanation:

- **First loop:** Calculate total usage.
- **If total > 6000,** identify how many minutes are billable.
- **Second loop:** For each usage session, apply the correct scheme (day/night) based on starting hour. Only bill minutes that are beyond the free quota, and stop charging once all extra minutes are billed.
- **Minute rounding:** CEIL used to round up partial minutes.

How the Design Meets the Scenario

- **Tables** are normalized, logically separated by entity, and ready for relational queries.
- **Eligibility logic** matches the policy exactly, including both CGPA and disciplinary count.
- **Billing function** applies correct scheme, minute rounding, and partial session billing across free/paid threshold.
- **PL/SQL functions** can be called from applications or triggers as needed.

Summary of Steps

1. Identify and design all entities/tables needed, ensuring normalization and referential integrity.
2. For eligibility, write a function that checks CGPA and counts disciplinary records in the past 6 months.
3. For billing, sum internet usage for the month, subtract free quota, and apply rates by session start time and minute rounding for excess usage.
4. Code with comments for clarity and maintainability.

Let me know if you want to see sample data or need ER diagrams!

Solve it in details with clear explanation of everything.
explain everything to me and the steps to solve it

Question 3

Additional Information:

The university introduces a scholarship having a total amount of "X US\$" which will be distributed among the students as per the following policy:

Scholarship Selection Policy:

Students having $CGPA \geq 3.25$ and having no disciplinary record will be initially selected. Then priority will be given based on CGPA only. Additionally, each student will receive an amount of "Y US\$" which will be materialized by another transaction similar to PA transaction. The process will continue as long as the fund is available. So, it cannot guarantee that all initially selected students will avail it.

Tasks:

- a) Write a procedure in PL/SQL to automate the Scholarship Disbursement Process as outlined.

b) What is the basic benefit of using %TYPE? Differentiate between implicit and explicit cursor.

Let's go step-by-step, explaining every detail and the reasoning behind each solution.

a) PL/SQL Procedure to Automate Scholarship Disbursement

Problem Breakdown

- Total Fund = X US\$
- Per Student Scholarship = Y US\$
- Selection Criteria:
 - $CGPA \geq 3.25$
 - No disciplinary record (i.e., student does not exist in Disciplinary_Record table)
- **Priority:** Higher CGPA gets priority
- **Process:**
 - Select eligible students sorted by CGPA descending
 - Disburse to as many as possible until the fund (X) is exhausted
 - Each transaction should be recorded, similar to PA_Transaction (assume a table Scholarship_Transaction)

Step 1: Prepare Required Table

Suppose you have a table for recording scholarship transactions:

sql

```
CREATE TABLE Scholarship_Transaction (  
  Transaction_ID NUMBER PRIMARY KEY,  
  Student_ID NUMBER NOT NULL,  
  Amount NUMBER(6,2) NOT NULL,
```



```
Transaction_Date DATE NOT NULL,  
FOREIGN KEY (Student_ID) REFERENCES Student(Student_ID)  
);
```

Step 2: The Procedure Logic

Algorithm Steps

1. Select all students where CGPA ≥ 3.25 and **not** in Disciplinary_Record table.
2. Sort students by CGPA descending.
3. Loop through students, for each:
 - If enough fund remains (Fund_Remaining $\geq Y$), insert transaction and decrease the fund.
 - Else, stop the process.
4. End when fund is insufficient or all students have received scholarship.

PL/SQL Procedure

```
sql  
  
CREATE OR REPLACE PROCEDURE Disburse_Scholarship(  
  p_Total_Fund IN NUMBER, -- X US$  
  p_Per_Student IN NUMBER, -- Y US$  
  p_Transaction_Date IN DATE -- Date of the disbursement  
)  
IS  
  CURSOR eligible_cur IS  
    SELECT s.Student_ID, s.CGPA  
    FROM Student s  
    WHERE s.CGPA >= 3.25  
    AND NOT EXISTS (  
      SELECT 1 FROM Disciplinary_Record d  
      WHERE d.Student_ID = s.Student_ID  
    )  
    ORDER BY s.CGPA DESC;  
  
  v_Fund_Remaining NUMBER := p_Total_Fund;  
  v_New_Transaction_ID NUMBER;  
BEGIN  
  FOR stu_rec IN eligible_cur LOOP
```

```

EXIT WHEN v_Fund_Remaining < p_Per_Student;

-- Get next transaction ID (simple way: max + 1)
SELECT NVL(MAX(Transaction_ID), 0) + 1 INTO v_New_Transaction_ID FROM
Scholarship_Transaction;

-- Insert scholarship transaction
INSERT INTO Scholarship_Transaction (
    Transaction_ID, Student_ID, Amount, Transaction_Date
) VALUES (
    v_New_Transaction_ID, stu_rec.Student_ID, p_Per_Student, p_Transaction_Date
);

-- Deduct fund
v_Fund_Remaining := v_Fund_Remaining - p_Per_Student;
END LOOP;
COMMIT;
END;
/

```

Explanation of Steps

- The **cursor** fetches all eligible students in priority order.
- For each, it checks if the fund can cover at least one more scholarship.
- If yes, it records the transaction and updates the fund balance.
- The loop **exits automatically** if the fund is insufficient for another student.
- **Transaction_ID** is set by taking the max and adding one (this is a simple approach; sequences are better in production).
- **COMMIT** at the end finalizes the disbursement.

b) Benefits of %TYPE and Difference Between Implicit and Explicit Cursor

1. Benefit of %TYPE

- **%TYPE** is a PL/SQL attribute that allows a variable or parameter to have **the same data type as a column in a table** or another variable.

- **Basic Benefit:**
 - Ensures that if the column's datatype changes in the table, your variable will automatically match the new datatype, **reducing maintenance errors** and making code more robust and future-proof.
- **Example:**

```
sql

DECLARE
  v_CGPA Student.CGPA%TYPE;
BEGIN
  -- Now v_CGPA will always match the datatype of Student.CGPA
END;
```

2. Implicit vs. Explicit Cursor

Feature	Implicit Cursor	Explicit Cursor
Definition	Managed automatically by Oracle for single SQL DMLs	Declared and controlled explicitly by the programmer
Use Case	For simple, single-row queries (SELECT INTO, INSERT, UPDATE, DELETE)	For queries returning multiple rows (need to fetch row by row)
Control	Automatic open, fetch, close	Manual OPEN, FETCH, CLOSE commands needed
Attributes	Use SQL%ROWCOUNT, SQL%FOUND etc.	Use cursor_name%ROWCOUNT, cursor_name%FOUND etc.
Example	<code>SELECT ... INTO ...</code>	<code>CURSOR c1 IS SELECT ...; OPEN c1; FETCH c1 ...; CLOSE c1;</code>

Examples

- **Implicit Cursor Example:**

```
sql

DECLARE
  v_name VARCHAR2(50);
BEGIN
```

```
SELECT Name INTO v_name FROM Student WHERE Student_ID = 1;
-- Oracle handles cursor behind the scenes
END;
```

- **Explicit Cursor Example:**

```
sql

DECLARE
  CURSOR stu_cur IS SELECT Name FROM Student WHERE CGPA > 3.5;
  v_name VARCHAR2(50);
BEGIN
  OPEN stu_cur;
  LOOP
    FETCH stu_cur INTO v_name;
    EXIT WHEN stu_cur%NOTFOUND;
    -- process v_name
  END LOOP;
  CLOSE stu_cur;
END;
```

Summary Table

Concept	Description/Benefit
%TYPE	Binds variable's datatype to a table column for consistency
Implicit Cursor	Automatic, for single-row SQL, less control, less code
Explicit Cursor	Manual, for multi-row queries, more control, more code

Let me know if you want any part explained with more real-world analogies, or want a code walkthrough with sample data!